VEL TECH RANGARAJAN DR. SAGUNTHALA R&D INSTITUTE OF SCIENCE AND TECHNOLOGY
Chennai, India · Office of International Relations · Value Added Course

Reinforcement Learning and Language Model Alignment

VTR UGE 21

DAY 5 · MODULE 5

Direct Preference Optimization and Evaluation

Preference optimisation without a reward model or sampling, compared with reinforcement learning from human feedback (RLHF) on matched data, the evaluation of aligned models with judges, intervals and seeds, and a live dashboard of the whole project.

Prof. Dr. Utku Kose
Full Professor, Department of Computer Engineering, Süleyman Demirel University, Isparta, Türkiye
Founding Director, AI Application and Research Center (YAZEM), Süleyman Demirel University
Head of the Computer Science Division, Department of Computer Engineering, Süleyman Demirel University
Additional affiliations: University of North Dakota (USA), Universidad Panamericana (Mexico City, Mexico), Vel Tech University (Chennai, India)
IEEE Senior Member, ACM Professional Member

ORCID 0000-0002-9652-6415 · utkukose.com · github.com/utkukose
utkukose@sdu.edu.tr · utku.kose@und.edu · ukose@up.edu.mx · utkukose@gmail.com

Overview

Reinforcement learning from human feedback, abbreviated RLHF, needs a reward model, sampling during training and careful tuning. Direct preference optimisation, abbreviated DPO, reaches the same goal with a simple classification loss on the preference pairs themselves [1]. This day derives the idea and compares DPO with RLHF on the same fine-tuned model and the same preference pairs. It then turns to a question that is harder than it looks: how to tell whether one aligned model is better than another. It compares judges and their biases [2, 3], puts intervals around every win rate and asks how many comparisons and seeds a claim needs [4]. A live dashboard then gathers the project of the week on one page: The tables and the networks of the five days are evaluated again in the browser and checked against Python.

Estimated study time: 6 to 8 hours.

Learning outcomes

By the end of the day, students are expected to derive the loss of direct preference optimisation (DPO) from the optimum of the RLHF objective with its Kullback-Leibler (KL) penalty and from the Bradley-Terry model, to implement it and explain the role of its gradient weight, to compare DPO with RLHF on gold reward, KL divergence and the quality of the outputs and explain the drift of DPO for small values of its parameter beta, to evaluate aligned models with several judges and recognise a judge that measures a surface property, to treat ties and report bootstrap intervals, and to plan the number of comparisons and seeds needed for a decision. In Python, students are expected to distinguish an alias from a copy of an object, to compute with NumPy arrays of probabilities and to write a function that returns a dictionary of results.

Python in this day

The Colab notebook opens with Python step 5: Objects, copies and the DPO loss, right after its setup section. It covers how two names can refer to one object and why the reference policy must be a copy. NumPy arrays then compute the optimum of the lecture, and a function returns the margin, the loss and the gradient weight of a pair as a dictionary [5, 6]. After the step, the notebook continues with DPO and the evaluation.

How these notes point to the Python code. Every method of this lecture is also written as Python code in the Colab notebook of the day, the file `NB05_dpo_and_evaluation.ipynb`. A Colab notebook is a document of text and Python code that runs in the web browser, with nothing to install. It opens with the button Open in Colab on the day overview page of the course, https://github.com/utkukose/rl-alignment-veltech-NB-lecture/tree/main/days/day-05. The code of the notebook is divided into numbered sections. In these notes, each box titled *Python code in the Colab notebook, Section N* names the section whose code carries out what the text above the box explains, and says what to run and what to look at there. The animations that the notes mention are interactive figures on the lecture page of the day, https://utkukose.github.io/rl-alignment-veltech-NB-lecture/days/day-05/lecture.html. The interactive lab, with the practice parts and the self-assessment of the day, is at https://utkukose.github.io/rl-alignment-veltech-NB-lecture/days/day-05/lab.html. Each box titled *Interactive lab, Part X* opens one part of the lab. The part ends with a link back to the lecture page, and the notebook is always opened from the boxes of the lecture.

Python step 5: Objects, copies and the DPO loss

The reference policy must stay frozen while the policy changes, and the optimum and the loss of direct preference optimisation (DPO) follow from a few probabilities. This step shows both in plain Python and NumPy [5, 6].

An alias is not a copy

```
import copy
class Policy:
    def __init__(self, weights):
        self.weights = weights

pol = Policy([1.0, 2.0])
alias = pol                     # a second name for the same object
frozen = copy.deepcopy(pol)     # an independent object with the same content
pol.weights[0] = 99.0
print("alias :", alias.weights)
print("frozen:", frozen.weights)
```

Output

```
alias : [99.0, 2.0]
frozen: [1.0, 2.0]
```

Assignment in Python never copies an object; it gives the object another name. A reference policy created by `ref = pol` would change with every update of the policy, and the Kullback-Leibler (KL) divergence between the two would always be zero. `copy.deepcopy`, or the `copy` method of the language model in the notebook, creates an independent object.

The optimum with NumPy arrays

```
import numpy as np
p_ref = np.array([0.4, 0.3, 0.2, 0.1])   # probabilities under the reference
r = np.array([0.0, 0.5, 1.0, 2.0])       # rewards of the four sentences
for beta in (1.0, 0.25):
    w = p_ref * np.exp(r / beta)
    p_opt = w / w.sum()                  # divided by Z: the optimum
    reward = (p_opt * r).sum()
    kl = (p_opt * np.log(p_opt / p_ref)).sum()
    print(f"beta {beta}: optimum {p_opt.round(3)}, reward {reward:.2f}, KL {kl:.2f}")
```

Output

```
beta 1.0: optimum [0.184 0.227 0.25  0.339], reward 1.04, KL 0.26
beta 0.25: optimum [0.001 0.007 0.035 0.957], reward 1.95, KL 2.07
```

Arithmetic on NumPy arrays works element by element: `p_ref * np.exp(r / beta)` multiplies every reference probability by its own factor, and `w.sum()` is the number Z of the lecture. The KL divergence is the mean of $\log \pi^*(y) - \log \pi_{\text{ref}}(y)$ under the optimum, which for four sentences is a sum weighted by `p_opt`. With $\beta = 0.25$ almost all probability moves to the sentence with the highest reward: The expected reward rises from 1.04 to 1.95, and the KL divergence from 0.26 to 2.07.

A function that returns a dictionary

```
import pandas as pd

def sigmoid(u):
    return 1 / (1 + np.exp(-u))

def dpo_pair(lw, ll, lw_ref, ll_ref, beta=0.1):
    """Margin, DPO loss and gradient weight of one pair."""
    z = (lw - lw_ref) - (ll - ll_ref)
    loss = -np.log(sigmoid(beta * z))
    return {"margin": z, "loss": loss, "weight": sigmoid(-beta * z)}

pairs = {"ranked correctly": (-5.0, -15.0, -10.0, -10.0),
         "ranked wrongly": (-15.0, -5.0, -10.0, -10.0)}
rows = [dpo_pair(*logps) for logps in pairs.values()]
print(pd.DataFrame(rows, index=list(pairs)).round(3))
```

Output

```
                  margin   loss  weight
ranked correctly    10.0  0.313   0.269
ranked wrongly     -10.0  1.313   0.731
```

`dpo_pair` computes the margin z, the loss $-\log \sigma(\beta\, z)$ and the gradient weight $\sigma(-\beta\, z)$ of the lecture and returns them as a dictionary, so that every number keeps its name. In `dpo_pair(*logps)` the star unpacks the tuple into the four log-probabilities, and a list of dictionaries with the same keys becomes a data frame with one row per pair. In both pairs the reference gives each sentence the log-probability -10, so the margin depends only on the policy. The pair that the policy ranks wrongly has the larger loss and pulls almost three times as hard.

Quick check. What would go wrong if the reference policy of reinforcement learning from human feedback (RLHF) were created with `ref = pol`?

From the optimum to DPO

> **TokenWorld so far**
>
> Days 1 to 3 worked on the small TokenWorld of four tokens. Day 1 solved it exactly, Day 2 learned its values from experience, and Day 3 learned the probabilities of its tokens with policy gradients. Day 4 enlarged it to a grammar of sentences and aligned a small language model in three stages. Supervised fine-tuning, abbreviated SFT, trained the model on a corpus, a collection of example sentences. A reward model learned to score sentences from pairs that an annotator, the person who labels the data, had compared [7]. RLHF then raised this score with a policy gradient method, while a penalty kept the model close to the fine-tuned one [8, 9]. Without the penalty, the policy hacked the reward model and ended up writing a single sentence.
>
> This day keeps the large TokenWorld, the fine-tuned model and the same pairs. It replaces the last two stages by a single one and then asks how the result of an alignment should be measured, with judges, intervals and seeds. The final project of the course uses both.



Figure. The last stage of the project: alignment without a reward model, and the evaluation of aligned models.

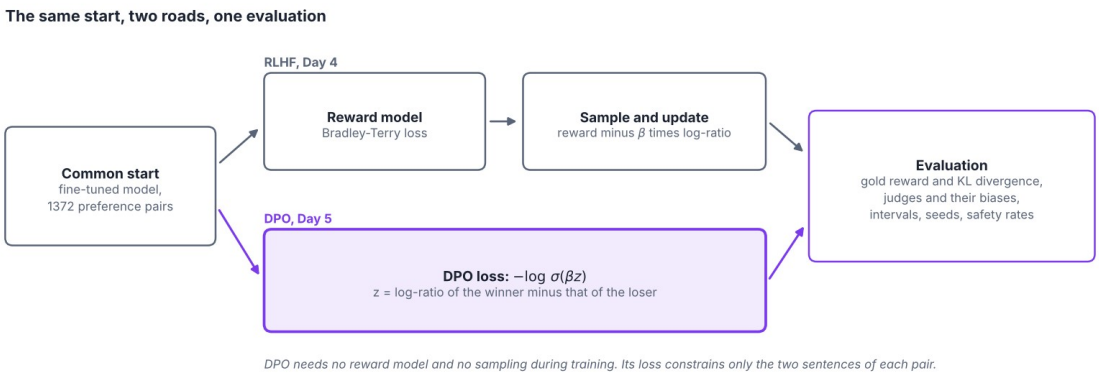


Figure. The plan of the day: RLHF and DPO start from the same fine-tuned model and the same pairs, take different roads and meet again in the evaluation.

The world is the large TokenWorld of Day 4. A sentence consists of a subject, a verb, a place and, in 45 percent of the cases, an extra phrase, for example `the cat waited near the door`. Each part has six choices, so the grammar contains 6 × 6 × 6 × 7 = 1512 sentences of 6 to 9 words. The quality of a sentence is measured by the gold reward of Day 4, which stands for what people want. No training method sees it directly: It reaches them only through the choices of an annotator, and its main use is to measure the result.

The gold reward favours a good opening word and calm words, and it penalises loud words, words beyond the sixth and a repeated content word. A content word is any word except `the`, `a`, `my`, `in`, `on`, `by`, `at`, `near`, `under`, `beside`, `for` and `while`. For example, `the cat waited under the tree quietly` opens with `the` and contains the three calm words `waited`, `under` and `quietly`, but it has seven words: It earns 1.0 + 3 × 0.45 − 0.10 = 2.25, the highest gold reward in the grammar. The sentence `the dog sang near the door` contains the loud word `sang` and earns 1.0 − 0.45 = 0.55.

A language model gives a probability to every possible next token, given the tokens so far, which makes it a policy: The sentence so far is the state and the next token is the action. The policy is written π_θ, where θ stands for the parameters, the numbers inside the model that training changes. The probability of a sentence y with the tokens y_1, …, y_T, the end token included, is the product of the probabilities of its tokens, so its logarithm is a sum:

$$\log \pi_\theta(y) = \sum_{t=1}^{T} \log \pi_\theta(y_t \mid y_1, \ldots, y_{t-1})$$

This sum is the log-probability of the sentence. The sign Σ adds the terms for $t = 1, \ldots, T$, the vertical bar reads *given*, and log is the natural logarithm throughout the day. A real language model also depends on a prompt x and is written $\pi_\theta(y \mid x)$, but the small world has no prompt.

After fine-tuning on the 4000 sentences of the corpus, the model writes sentences of which 98.6 percent follow the grammar, with a mean gold reward of 1.12. A copy of this model is the starting point of every

alignment method, and the unchanged original is the reference policy π_{ref}, from which the distance of the new policy is measured.

The preference data are 1500 pairs of sentences sampled from the fine-tuned model, which means written by it at random with its own probabilities. A simulated annotator who follows the gold reward picks the better sentence of each pair and errs in 10 percent of the cases. Pairs with equal gold reward are skipped, which leaves 1372 pairs, 141 of them with a flipped label. The chosen sentence of a pair is the winner y_w and the other one the loser y_l. The reward model r_φ is fitted to 1166 of the pairs and agrees with the annotator on 88.3 percent of the remaining 206.

> **Python code in the Colab notebook, Section 1.**
>
> Open Section 1 of the Colab notebook and run its six cells. They rebuild the world of Day 4: the corpus with the gold reward, the language model `NeuralLM`, its fine-tuning by `train_sft`, the preference pairs from `make_pairs`, the reward model from `train_rm` and the function `rlhf`. The cells print the same lines as on Day 4. The fine-tuned model `sft`, the pairs `Wp` and `Lp` and the reward model `rm` are used by every later section.

The distance between the new policy and the reference is measured by the Kullback-Leibler divergence, abbreviated KL divergence: the average difference of the two log-probabilities over sentences written by the new policy. It is estimated from n sampled sentences $y^{(1)}, \ldots, y^{(n)}$:

$$D_{\text{KL}}(\pi_\theta \,\|\, \pi_{\text{ref}}) = \mathbb{E}_{y\sim\pi_\theta}\left[\log \pi_\theta(y) - \log \pi_{\text{ref}}(y)\right]$$

$$\approx \frac{1}{n}\sum_{i=1}^{n}\left[\log \pi_\theta(y^{(i)}) - \log \pi_{\text{ref}}(y^{(i)})\right]$$

The sign E with the subscript $y \sim \pi_\theta$ is the average over sentences y drawn from the policy, and $\approx$ reads *is approximately*. The divergence is zero when the two policies are equal. A value of 5 means that a typical sentence of the new policy is e^5, about 150, times more probable under the new policy than under the reference.

RLHF searches for the policy that earns a high reward without moving far from the reference. Its objective J, the quantity that it maximises, is the expected reward minus the KL divergence, weighted by a positive number β:

$$J(\pi_\theta) = \mathbb{E}_{y\sim\pi_\theta}\left[r(y)\right] - \beta\, D_{\text{KL}}(\pi_\theta \,\|\, \pi_{\text{ref}})$$

Here $r(y)$ is the reward of the sentence y, in RLHF the score $r_\varphi(y)$ of the reward model. A large β keeps the policy close to the reference, and a small β lets it move far for a little more reward. The objective of RLHF, reward minus a KL penalty, has a known optimum: The best policy is the fine-tuned model reweighted by the exponential of the reward [1, 10].

$$\pi^*(y) = \frac{1}{Z}\,\pi_{\text{ref}}(y)\,\exp\!\left(\frac{r(y)}{\beta}\right)$$

$$Z = \sum_{y'} \pi_{\text{ref}}(y')\,\exp\!\left(\frac{r(y')}{\beta}\right)$$

The star marks the best policy, and $\exp(u)$ is another way to write e^u. Every sentence keeps its reference probability, multiplied by a factor that grows with its reward. The number Z is the sum of these products over all sentences y', and dividing by it makes the probabilities add up to one.

Worked example: the optimum in a world of three sentences

A world has the three sentences A, B and C. The reference writes them with the probabilities 0.5, 0.3 and 0.2, and their rewards are 0, 1 and 2. With $\beta = 1$ the factors $e^{r/\beta}$ are $e^0 = 1$, $e^1 = 2.718$ and $e^2 = 7.389$. The products with the reference probabilities are $0.5 \times 1 = 0.500$, $0.3 \times 2.718 = 0.815$ and $0.2 \times 7.389 = 1.478$, and their sum is $Z = 0.500 + 0.815 + 1.478 = 2.793$. The optimum is $0.500 / 2.793 = 0.179$, $0.815 / 2.793 = 0.292$ and $1.478 / 2.793 = 0.529$: Probability has moved from A to C.

The objective confirms the result. The reference itself earns the expected reward $0.5 \times 0 + 0.3 \times 1 + 0.2 \times 2 = 0 + 0.3 + 0.4 = 0.70$ and pays no penalty, so $J = 0.70$. A policy that always writes C earns the reward 2 and has the divergence $1 \times \log(1 / 0.2) = \log 5 = 1.61$, so $J = 2 - 1.61 = 0.39$. The optimum earns the expected reward $0.179 \times 0 + 0.292 \times 1 + 0.529 \times 2 = 0.292 + 1.058 = 1.350$. Its divergence adds $\pi^*(y) \log(\pi^*(y) / \pi_{\text{ref}}(y))$ over the three sentences: $0.179 \times (-1.027) + 0.292 \times (-0.027) + 0.529 \times 0.973 = -0.184 - 0.008 + 0.515 = 0.323$. So $J = 1.350 - 0.323 = 1.03$, more than both.

With $\beta = 0.5$ the factors are $e^0 = 1$, $e^2 = 7.389$ and $e^4 = 54.598$, the products 0.500, $0.3 \times 7.389 = 2.217$ and $0.2 \times 54.598 = 10.920$, their sum 13.637, and the optimum $0.500 / 13.637 = 0.037$, $2.217 / 13.637 = 0.163$ and $10.920 / 13.637 = 0.801$: A smaller β concentrates the policy on the sentence with the highest reward.

Rafailov and colleagues turned this around [1]. If the optimal policy determines the reward, the reward can be written in terms of the policy. Taking the logarithm of the optimum and solving for the reward gives

$$r(y) = \beta \log \frac{\pi^*(y)}{\pi_{\text{ref}}(y)} + \beta \log Z$$

Read in this direction, the formula gives every policy π_θ a reward for which it would be the optimum: $\beta \log(\pi_\theta(y) / \pi_{\text{ref}}(y))$ plus a constant. This is the implicit reward of the policy. It is high for a sentence whose probability the policy has raised far above the reference. In the worked example with $\beta = 1$, the implicit rewards of A, B and C are $\log(0.179 / 0.5) = \log 0.358 = -1.03$, $\log(0.292 / 0.3) = \log 0.973 = -0.03$ and $\log(0.529 / 0.2) = \log 2.645 = 0.97$: the true rewards 0, 1 and 2 minus the constant $\log Z = \log 2.793 = 1.03$.

The Bradley-Terry model of Day 4 connects rewards with preferences [11]: The probability that the annotator prefers y_w to y_l is the logistic function of the difference of their rewards.

$$P(y_w \succ y_l) = \sigma\left(r(y_w) - r(y_l)\right)$$

$$\sigma(u) = \frac{1}{1 + e^{-u}}$$

The sign $\succ$ reads *is preferred to*, and σ is the logistic function of Day 4. The logistic function turns any number u into a number between 0 and 1, with $\sigma(0) = 0.5$. When the implicit rewards of the policy are put into this model, the constant $\beta \log Z$ cancels in the difference. What remains is β times the margin z of the pair:

$$z = \log \frac{\pi_\theta(y_w)}{\pi_{\text{ref}}(y_w)} - \log \frac{\pi_\theta(y_l)}{\pi_{\text{ref}}(y_l)}$$

The first term says how far the policy has raised the log-probability of the winner above the reference, and the second term says the same for the loser. The margin is zero as long as the policy equals the reference, and positive when the policy has moved towards the winner. A reward model is trained by making the observed preferences as probable as possible. DPO does the same with the policy itself: It minimises the negative logarithm of the Bradley-Terry probability of each pair, the DPO loss [1].

$$\mathcal{L}_{\mathrm{DPO}}(\theta) = -\log \sigma(\beta z)$$

Direct preference optimisation trains the model on the preference pairs with this loss, without a separate reward model and without sampling new answers during training. Its parameter beta plays the role of the KL weight. The loss is called a classification loss because it is the loss of a classifier with two classes: The policy has to tell the winner of a pair from the loser. The loss is close to zero when $\beta\, z$ is large and positive, it is log 2 = 0.693 at $z = 0$, and it grows almost in a straight line for negative margins.

> **In everyday life: learning from the choices directly**
>
> A cook who wants to please the guests can first write a scoring rule for dishes from their comments and then cook for the rule, which is the road of RLHF. Or the cook can change the recipe directly towards the dish that the guests chose in each tasting, without any rule in between, which is the road of DPO. The second road is shorter, but it learns only about the dishes that were actually tasted.

> **Worked example: the DPO loss of two pairs**
>
> Let $\beta = 0.1$. The policy gives the winner of a pair the log-probability −9.0 and the loser −12.0, and the reference gives them −10.5 and −8.0. Compared with the reference, the log-probability of the winner has changed by −9.0 − (−10.5) = 1.5 and that of the loser by −12.0 − (−8.0) = −4.0, so the margin is $z = 1.5 - (-4.0) = 5.5$. Then $\beta\, z = 0.1 \times 5.5 = 0.55$, $\sigma(0.55) = 1 / (1 + e^{-0.55}) = 1 / (1 + 0.577) = 0.634$, and the loss is −log 0.634 = 0.456.
>
> In a second pair the policy has moved the wrong way, and $z = -8$. Then $\beta\, z = 0.1 \times (-8) = -0.8$, $\sigma(-0.8) = 1 / (1 + e^{0.8}) = 1 / (1 + 2.226) = 0.310$, and the loss is −log 0.310 = 1.171. The gradient weights of the two pairs, defined in the next paragraph, are $\sigma(-0.55) = 1 - 0.634 = 0.366$ and $\sigma(0.8) = 1 - 0.310 = 0.690$: The pair that is ranked wrongly pulls almost twice as hard.

Training lowers the loss by gradient descent: Every parameter is moved a small step against the slope of the loss. The gradient ∇_θ, written with the sign nabla, collects these slopes for all parameters. For one pair it is

$$\nabla_\theta \mathcal{L}_{\mathrm{DPO}} = -\beta\, q\, [\nabla_\theta \log \pi_\theta(y_w) - \nabla_\theta \log \pi_\theta(y_l)]$$

$$q = \sigma(-\beta z)$$

The reference does not depend on θ and has dropped out. A step against this gradient raises the log-probability of the winner and lowers that of the loser, and the gradient weight q sets its strength. The weight is 0.5 at $z = 0$, close to 1 for a pair that the policy ranks wrongly by a wide margin, and close to 0 for a pair that it already ranks correctly. DPO therefore spends its effort on the pairs that it has not learned yet.

In the project, the policy starts as a copy of the fine-tuned model. One pass through all pairs is called an epoch. In each of 8 epochs the 1372 pairs are shuffled and split into 21 mini-batches of 65 or 66 pairs. For each mini-batch the parameters take one step against the mean gradient of its B pairs:

$$\theta \leftarrow \theta - \alpha\, \frac{1}{B} \sum_{i=1}^{B} \nabla_\theta \mathcal{L}_{\mathrm{DPO}}^{(i)}$$

The arrow means that the right-hand side becomes the new value of θ, and α is the learning rate, which sets the size of the step and is 10^{-3} here. The step is taken with the Adam rule, a form of gradient descent that adapts the step of every parameter to the recent sizes of its gradient. With $\beta = 0.1$ the training makes 8 × 21 = 168 steps and takes about two seconds.

Interactive lab, Part A.

The formulas of this section solved step by step with numbers for one pair: the margin, the implicit rewards, the logistic function, the loss of DPO, its gradient weight and one gradient step. Sliders set the four log-probabilities and β, and a figure shows both curves. Part A opens at https://utkukose.github.io/rl-alignment-veltech-NB-lecture/days/day-05/lab.html#pa.

Animation on the lecture page: The DPO loss $-\log \sigma(\beta\, z)$ and the gradient weight $\sigma(-\beta\, z)$ against the margin z of a pair. Change β and move the margin.

With $\beta = 0.1$, the animation shows a loss of 2.13 and a weight of 0.88 for a pair at $z = -20$, and a loss of 0.13 and a weight of 0.12 at $z = 20$. A smaller β stretches both curves: With $\beta = 0.02$ a pair at $z = 40$ still has a weight of 0.31, so the policy is pushed on long after it ranks the pair correctly.

Python code in the Colab notebook, Section 2.

Open Section 2 of the Colab notebook and run it. The function `dpo` is the training loop of this section. Its variable `z` holds the product $\beta\, z$ of the formulas, `q` is the gradient weight, and the two calls of `grad_weighted` raise the winners and lower the losers. The cell prints one row per epoch with the gold reward, the KL divergence, the share of sentences in the grammar and the column `pair_accuracy`.

Check your understanding (1). What does DPO need that RLHF also needs?

A. A separate reward model

B. Sampling during training

C. Preference pairs

D. A value function

Check your understanding (2). With $\beta = 0.5$, the policy has raised the log-probability of the winner of a pair by 1 and that of the loser by 3, both compared with the reference. What are the margin z and the gradient weight of the pair?

A. $z = 2$, weight about 0.27

B. $z = -2$, weight about 0.73

C. $z = -2$, weight about 0.27

D. $z = 4$, weight about 0.12

DPO against RLHF on matched data

A fair comparison gives both methods the same fine-tuned model and the same pairs. Such a comparison is called matched: Whatever differs in the results then comes from the methods and not from their data. RLHF uses the pairs through its reward model. In each of 300 iterations it samples 64 sentences from its policy, and each sentence receives the score of the reward model minus β times its own distance from the reference, with $\beta = 0.2$:

$$\tilde{r}(y) = r_\phi(y) - \beta\,[\log \pi_\theta(y) - \log \pi_{\text{ref}}(y)]$$

The policy gradient step of Day 3 then raises the probability of the sentences whose penalised reward $\tilde{r}$, written with a tilde, lies above the mean of the batch. DPO trains as described above, with $\beta = 0.1$. The

amount of computation is similar: RLHF processes 300 × 64 = 19200 sentences and DPO 8 × 1372 × 2 = 21952, eight passes over two sentences per pair.

After training, 512 sentences are drawn from each policy, and five numbers describe them. They are the mean gold reward, the estimate of the KL divergence from the fine-tuned model, the mean number of words, the grammaticality and the number of different sentences, a measure of variety. The grammaticality is the share of sentences that belong to the 1512 sentences of the grammar.

Policy	Gold reward	KL divergence	Words	In the grammar	Different sentences
Fine-tuned model (SFT)	1.117	0	6.81	99.2 %	376
RLHF, $\beta = 0.2$	2.069	2.74	6.60	98.6 %	103
DPO, $\beta = 0.1$	2.234	5.09	7.21	90.8 %	101

In this experiment, DPO reaches a slightly higher gold reward than RLHF but moves further from the fine-tuned model, and a clearly smaller share of its sentences still follows the grammar. Its KL divergence is almost twice that of RLHF, and 9.2 percent of its sentences lie outside the grammar, against 1.4 percent for RLHF. Both aligned policies have lost variety: They write about 100 different sentences where the fine-tuned model writes 376. DPO needs about half the training time of RLHF, 1.9 against 4.0 seconds in one run.

A typical sentence of DPO outside the grammar is `the dog waited beside the garden quietly`: The corpus contains `beside the lamp` and `in the garden`, and never `beside the garden`. The gold reward does not object, because it checks the opening word, the calm and the loud words, the length and repetition, and not the grammar. With the four calm words `waited`, `beside`, `garden` and `quietly` the sentence earns 1.0 + 4 × 0.45 − 0.10 = 1.0 + 1.80 − 0.10 = 2.70, more than the 2.25 of the best sentence of the grammar. Such a movement of the policy towards sentences that the corpus never contained is called drift. The gold reward alone would hide it, and the KL divergence and the grammaticality reveal it.

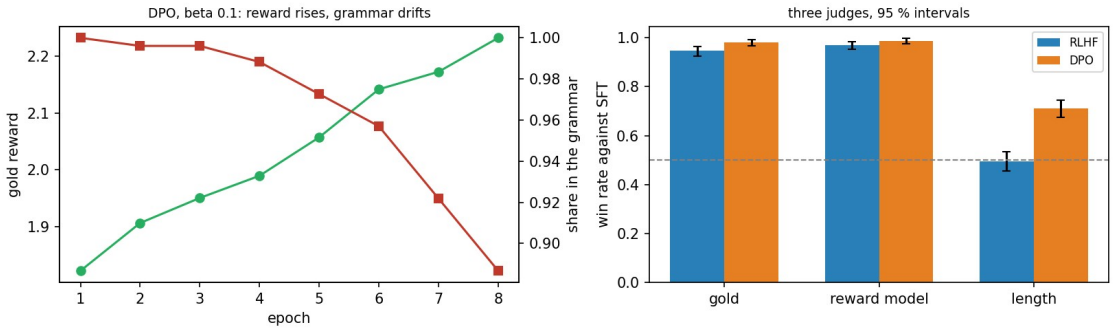


Figure. Left: DPO with β = 0.1 over eight epochs. The gold reward (circles, left axis) rises while the share of sentences in the grammar (squares, right axis) falls. Right: Win rates of RLHF and DPO against the fine-tuned model under three judges, with 95 percent intervals.

The left panel follows the DPO run epoch by epoch, each point measured on 256 sampled sentences. The gold reward rises from 1.82 after the first epoch to 2.23 after the eighth. The share of sentences in the grammar stays above 98 percent for four epochs and then falls to 89 percent, while the KL divergence grows from 1.96 to 4.91. A further number, the pair accuracy, is the share of positive margins in the last mini-batch of each epoch. It stays below 0.9, because one label in ten was flipped by the annotator and a policy that follows the gold reward contradicts those labels. The right panel belongs to the next section.

With a smaller beta, DPO drifts so far that most sentences break the grammar. The next table shows five runs of DPO after their eighth epoch, each measured on 256 sentences.

β of DPO	Gold reward	KL divergence	In the grammar
0.02	2.166	12.37	11 %
0.05	2.224	5.87	77 %
0.1	2.232	4.91	89 %
0.3	2.111	2.97	96 %
1.0	1.760	0.95	98 %

With $\beta = 0.02$ the KL divergence reaches 12.4 and only 11 percent of the sentences remain in the grammar, while the gold reward stays high. With $\beta = 1$ the policy hardly leaves the reference and gains much less. With $\beta = 0.3$, DPO ends close to RLHF with $\beta = 0.2$, at a KL divergence of 3.0 against 2.7: The same value of β does not mean the same distance in the two methods. Neither method is better in every respect; the comparison depends on what is measured.

Two properties of the DPO loss explain the drift. The loss depends on the product $\beta\, z$, so a small β demands a large margin: The weight of a pair falls to 0.12 at $z = 20$ with $\beta = 0.1$, and only at $z = 100$ with $\beta = 0.02$. And the loss constrains nothing but the two sentences of each pair, so the probability of every other sentence is free to change. RLHF charges its penalty on the sentences that the policy writes at that moment, so every new habit is charged at once.

Variants of DPO change the loss or the data in order to limit such effects. Identity preference optimisation, abbreviated IPO, replaces the logarithmic loss by the squared distance of the margin from a fixed target [12]:

$$\mathcal{L}_{\mathrm{IPO}}(\theta) = \left(z - \frac{1}{2\beta} \right)^2$$

The DPO loss can always be lowered further by a larger margin. The IPO loss is smallest at $z = 1 / (2\beta)$, which is $1 / (2 \times 0.1) = 1 / 0.2 = 5$ for $\beta = 0.1$, and rises again beyond this value. Kahneman-Tversky optimisation, abbreviated KTO, learns from single answers that are labelled as good or bad and needs no pairs [13].

> **Python code in the Colab notebook, Section 3.**
>
> Open Section 3 of the Colab notebook and run it. The function `summary` draws 512 sentences from a policy and computes the five numbers of the table above. The cell prints the table for the three policies, and its last loop repeats `dpo` for four other values of `beta`.

> Check your understanding (3). DPO reaches a higher gold reward than RLHF but breaks the grammar more often. What follows?
>
> A. DPO is better
>
> B. RLHF is better
>
> C. The answer depends on which property matters for the application
>
> D. Both failed

Judges and their biases

Aligned models are compared by asking a judge which of two answers is better, and reporting a win rate. The judge may be a person, a reward model or another language model [2]. For one comparison of an answer a of model A with an answer b of model B, the verdict of the judge is turned into a score s:

$$s = \begin{cases} 1 & \text{if the judge prefers } a \\ 0.5 & \text{if the judge sees no difference: a tie} \\ 0 & \text{if the judge prefers } b \end{cases}$$

The win rate of A against B is the mean score of n comparisons:

$$\hat{p} = \frac{1}{n} \sum_{i=1}^{n} s_i = \frac{\text{wins} + 0.5 \times \text{ties}}{n}$$

The hat marks an estimate, as on Day 1. A win rate above one half favours A, and a win rate of one half means that the judge sees no difference between the models. Counting a tie as half a win is a rule, and other rules exist: The strictest one counts ties as losses of A.

> **Worked example: a win rate under two rules for ties**
>
> Model A meets model B in 200 comparisons. The judge prefers A 96 times, prefers B 44 times and sees no difference 60 times. With ties counted as half a win, the win rate is (96 + 0.5 × 60) / 200 = (96 + 30) / 200 = 126 / 200 = 0.63. With ties counted as losses it is 96 / 200 = 0.48.
>
> The first number says that A is ahead, which fits 96 wins against 44 losses. The second number lies below one half and suggests the opposite.

> **In everyday life: draws and judges**
>
> Chess counts a draw as half a point for each player, so the points of a match always add up to the number of games. Judges of talent shows and wine tastings know their biases well: A performer who appears first or last, or a wine served in a particular position, is remembered better. Competitions therefore state their rules for draws and change the order of the performers, and an evaluation of language models needs the same care.

Each judge of this section is built from a scoring function: The judge prefers the sentence with the higher score and declares a tie when the scores are equal. The gold judge scores with the gold reward, the reward-model judge with the score r_φ of the reward model, and the length judge with the number of words. RLHF and DPO each meet the fine-tuned model in 400 comparisons, each with a fresh sentence from both models. The table gives the win rates with ties counted as half, and the 95 percent intervals that the next section explains.

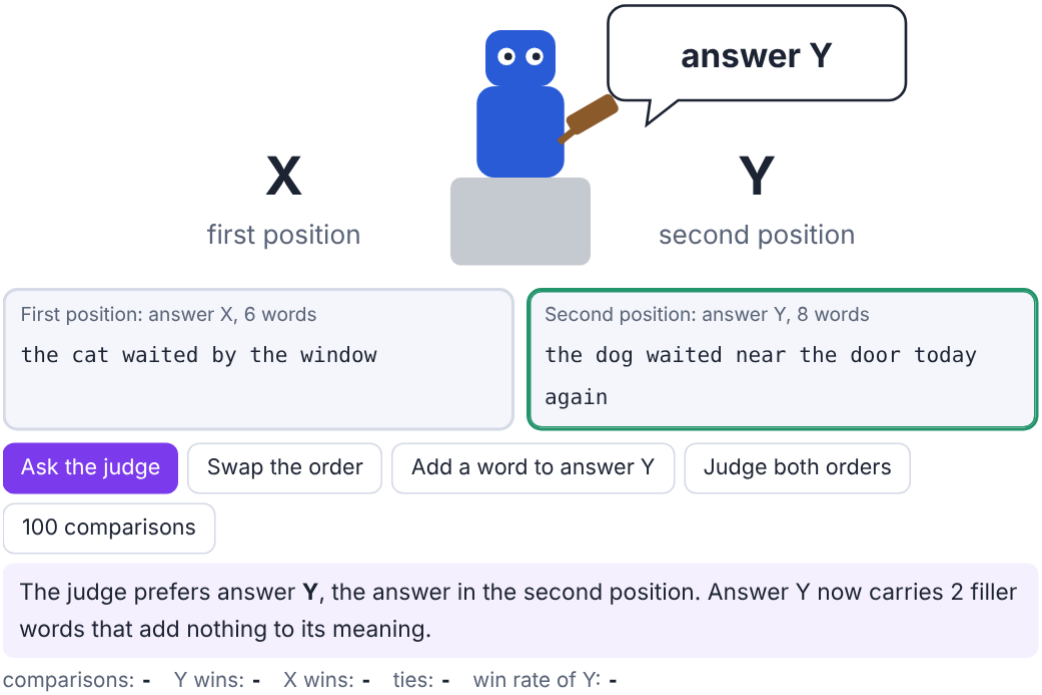
Judge	RLHF against the fine-tuned model	DPO against the fine-tuned model
Gold reward	0.948, interval 0.926 to 0.965	0.980, interval 0.966 to 0.991
Reward model	0.970, interval 0.952 to 0.985	0.988, interval 0.975 to 0.998
Number of words	0.496, interval 0.456 to 0.534	0.711, interval 0.674 to 0.746

Under the gold judge both methods clearly beat the fine-tuned model. The reward-model judge agrees and is even more favourable. For RLHF it is no independent judge: The policy was trained to raise exactly

this score, so a high win rate under it only shows that the training reached its own target. For DPO the reward model took no part in the training, but it has learned from the same pairs. An independent judge is one that shares neither the training nor its data with the models.

Judges have biases; a well-known one is a preference for longer answers [3]. A bias is a steady preference for something other than quality. In this experiment, a judge that prefers longer sentences cannot separate RLHF from the fine-tuned model, yet declares DPO clearly better, because DPO writes longer sentences: 7.2 words on average against 6.8. A judge that prefers short sentences ranks DPO below the fine-tuned model. Length is a surface property, a feature that can be read off an answer without understanding it, and a judge that responds to a surface property can carry an entire result. A length control removes this influence, for example by comparing only answers with the same number of words.

Language models that serve as judges also show a position bias: They tend to favour the answer in a certain position, often the first one, whatever it says [2]. The remedy is to judge every pair twice with the order swapped and to count a win only when both verdicts agree.



Interactive scene on the lecture page: A judge with habits. A judge with two habits added on purpose: It adds 0.3 to the score of the answer in the first position and 0.2 for every word. Two answers of equal quality wait for its verdict. Swapping their order, adding filler words to one of them and judging every pair in both orders show which habit each remedy removes. The button 100 comparisons pits a model against a copy of itself that pads six answers in ten with a filler word.

How ties are counted can also change a conclusion. Under the length judge, RLHF wins 123 of its 400 comparisons with the fine-tuned model, loses 126 and ties 151. With ties as half a win the win rate is (123 + 0.5 × 151) / 400 = 198.5 / 400 = 0.496, and with ties as losses it is 123 / 400 = 0.31, which reads like a clear defeat although wins and losses are balanced. Under the gold judge, RLHF against DPO has a win rate of 0.311 with ties as half and of 0.145 with ties as losses. Counting ties as half treats both models alike, because the win rates of A against B and of B against A then add up to one. Whatever the rule, it has to be stated with the win rate.

Python code in the Colab notebook, Section 4.

Open Section 4 of the Colab notebook and run it. The function `judge_from` turns a scoring function into a judge, and `win_rate` runs the 400 comparisons and computes the bootstrap interval. The cell prints the win rates of the table above with their intervals and the share of ties, and below it the win rate of RLHF against DPO under both rules for ties.

A cooking contest shows how strong a position bias can be. Dish A is slightly better than dish B, so that a jury without bias prefers it in 55 percent of the tastings. The jury, however, gives a bonus to the dish that it tastes first: With two dishes of equal quality it would prefer the first one in 70 percent of the tastings. The contest runs 400 tastings in each of three ways: with A always served first, with B always served first, and with the order swapped from one tasting to the next.

Animation on the lecture page: The win rate of dish A with its 95 percent interval after 400 tastings, for three ways of serving. The small dots show the expected win rates. Move the position bias of the jury and play the tastings again.

With A always served first, the jury prefers A in 74.0 percent of the tastings on average, and with B always served first in only 34.4 percent: The order of serving decides the verdict. In the first run, A wins 0.710 of the tastings in the first way, an interval from 0.666 to 0.754, and 0.385 in the second way, an interval from 0.337 to 0.433, so the two contests are decided for opposite dishes. Swapping the order removes the direction of the bias, as judging every pair in both orders does, but not its weight: The expected win rate is 0.542 instead of 0.55, because a bonus on both sides pulls every verdict towards a toss of a coin. The first run gives 0.542 with an interval from 0.494 to 0.591, undecided, and by the formula of the next section the contest would need more than 543 tastings. A jury that prefers the first dish in 90 percent of equal tastings leaves 0.518 and needs more than 2926.

Check your understanding (4). A judge prefers longer answers. What can it reward?

A. Better answers only

B. Answers that are merely longer

C. Shorter answers

D. Nothing

Intervals, sample size and seeds

A win rate measured on a few hundred comparisons is an estimate with an interval around it. Another 400 comparisons would give a slightly different number. A 95 percent interval is a range that is constructed so that it contains the true win rate in about 95 of 100 such experiments. The interval is obtained with the bootstrap, a method that imitates the repetition of the experiment with the data at hand. From the n scores of the experiment it draws n scores at random with replacement, so that some are drawn several times and others not at all, computes their mean, and does this 2000 times:

In everyday life: the margin of error of a poll

An opinion poll that asks 1000 people reports a share with a margin of error of about 3 points. Asking 4000 people halves the margin, as the formula of this section predicts, and asking only 250 doubles it. When two parties are 2 points apart and the margin is 3 points, the poll cannot tell which party leads, and a careful report says so.

$$\hat{p}^*_b = \frac{1}{n}\sum_{i=1}^{n} s^*_{b,i}, \qquad b = 1, 2, \ldots, 2000$$

The star marks a resampled quantity, and b numbers the resamples. The lower end of the interval is the 2.5th percentile of the 2000 means, the value below which 2.5 percent of them fall, and the upper end is the 97.5th percentile. When the interval contains one half, the comparison is undecided. When the whole interval lies above or below one half, the comparison is decided: The lab calls this decided at the 5 percent level, and statistics calls the difference significant.

For a calculation by hand, the interval is close to

$$\hat{p} \pm 1.96 \sqrt{\frac{\hat{p}\,(1-\hat{p})}{n}}$$

The sign ± reads *plus or minus*, and the term behind it is the half-width of the interval. The factor 1.96 belongs to 95 percent. The half-width is largest at $\hat{p} = 0.5$, where it equals $1.96 \times \sqrt{(0.5 \times 0.5)} / \sqrt{(n)} = 1.96 \times 0.5 / \sqrt{(n)} = 0.98 / \sqrt{(n)}$: 0.98 / 10 = 0.098 for 100 comparisons, 0.98 / 20 = 0.049 for 400 and 0.98 / 40 = 0.0245 for 1600. Four times as many comparisons halve it. The formula holds for comparisons without ties. Ties pull the scores together, so with many ties the bootstrap interval is narrower than the formula says, and the formula errs on the safe side.

> **Worked example: an interval by hand**
>
> The win rate 0.63 of the previous example came from 200 comparisons. Its half-width is $1.96 \sqrt{(0.63 \times 0.37 / 200)}$. Step by step, 0.63 × 0.37 = 0.2331, 0.2331 / 200 = 0.0011655, $\sqrt{(0.0011655)} = 0.0341$ and 1.96 × 0.0341 = 0.067. The interval runs from 0.63 − 0.067 = 0.563 to 0.63 + 0.067 = 0.697. It lies above one half: The comparison is decided in favour of A.
>
> With 50 comparisons and the same win rate, the half-width would be $1.96 \sqrt{(0.2331 / 50)} = 1.96 \sqrt{(0.004662)} = 1.96 \times 0.0683 = 0.134$, twice as large. The interval from 0.63 − 0.134 = 0.496 to 0.63 + 0.134 = 0.764 would contain one half, and the comparison would be undecided.

The same formula tells how many comparisons an evaluation needs, its sample size. A win rate at a distance d from one half can be decided when the half-width is smaller than d:

$$n > \left(\frac{0.98}{d}\right)^2$$

A win rate of 0.55 has $d = 0.05$ and needs about $(0.98 / 0.05)^2 = 19.6^2 = 384$ comparisons, and a win rate of 0.52 needs about $(0.98 / 0.02)^2 = 49^2 = 2401$, about 2400. These are minimum values: The estimate scatters around the true win rate, so with exactly this n only about half of all experiments reach a decision. Twice as many comparisons raise the share to about four in five.

> **Interactive lab, Part B.**
>
> The formulas of this and the previous section solved step by step with numbers: the score of a comparison, the win rate under both rules for ties, the half-width, the interval and the decision, and the number of comparisons that a decision would need. Sliders set the wins, the ties and the losses. Part B opens at https://utkukose.github.io/rl-alignment-veltech-NB-lecture/days/day-05/lab.html#pb.

Animation on the lecture page: The spread of win-rate estimates over 400 simulated experiments. Change the number of comparisons and the true win rate and read off how often an experiment reaches a decision.

The animation repeats an evaluation 400 times and draws the 400 estimated win rates as a histogram, a bar chart that counts the estimates in each range, with a dashed line at one half and a solid line at the true win rate. With 100 comparisons and a true win rate of 0.55, only 18 percent of the experiments reach a decision, with about 400 comparisons 54 percent and with 1000 comparisons 88 percent. With a true win rate of exactly one half, between 5 and 7 percent of the experiments still decide: These are false alarms, the price of a 95 percent interval.

> **Interactive lab, Part C.**
>
> The evaluation desk: 400 real outputs of four policies, judges to choose, either rule for ties and the bootstrap interval of every win rate. Part C opens at https://utkukose.github.io/rl-alignment-veltech-NB-lecture/days/day-05/lab.html#pc.

Two comparisons under the gold judge use 25, 100, 400 and 1600 sentences per model. RLHF against the fine-tuned model is decided at every size, because its win rate stays above 0.95, far from one half. With 25 comparisons RLHF wins all of them, and the bootstrap interval collapses to the single point 1.000, because every resample of 25 wins consists of 25 wins. A rule of thumb fills the gap: After n wins in n comparisons, the true share of losses lies below about $3 / n$ with 95 percent confidence, here below 12 percent.

The second comparison sets two RLHF policies against each other that were trained by the same recipe with different seeds, the numbers that start the random number generator. With 25, 100 and 400 comparisons the two runs cannot be told apart. With 1600 comparisons the win rate is 0.520 and the interval, 0.501 to 0.537, lies barely above one half, so the comparison counts as decided although nothing in the recipe differs. This is weak evidence, since a 95 percent interval gives a false alarm in about one experiment out of twenty. Fresh sentences settle the matter: Repeated with other samples, the comparison is undecided in almost every case, and with 20000 comparisons the win rate is 0.50.

One interval is therefore never the last word, and a difference between two runs can come from the seed as well as from the method. In RLHF the seed decides which sentences are sampled during training, and in DPO it decides the order of the pairs. Results in reinforcement learning often vary strongly from seed to seed, so a comparison of two methods needs several runs of each. For the results $x_1, \ldots, x_m$ of m seeds, for example m win rates, the interval over seeds is

$$\bar{x} \pm t_m \frac{\mathrm{SD}}{\sqrt{m}}, \qquad \mathrm{SD} = \sqrt{\frac{1}{m-1} \sum_{i=1}^{m} (x_i - \bar{x})^2}$$

Here $\bar{x}$, written with a bar, is the mean of the m results, and SD is their standard deviation, the measure of spread of Day 1, here with $m - 1$ in the denominator because it is estimated from few values. The factor t_m replaces 1.96 when m is small: It is 4.30 for three seeds, 2.78 for five and 2.26 for ten. Three seeds with the win rates 0.62, 0.66 and 0.70 have the mean $(0.62 + 0.66 + 0.70) / 3 = 1.98 / 3 = 0.66$. Their distances from the mean are −0.04, 0 and 0.04, so $\mathrm{SD} = \sqrt{((0.0016 + 0 + 0.0016) / 2)} = \sqrt{(0.0016)} = 0.04$. The interval is $0.66 \pm 4.30 \times 0.04 / \sqrt{(3)} = 0.66 \pm 0.172 / 1.732 = 0.66 \pm 0.10$. A claim that one model is better needs its interval, its number of comparisons and more than one seed [4].

> **Interactive lab, Part D.**
>
> Six reported win rates with their intervals, each to be read as decided for A, decided for B or undecided. Part D opens at https://utkukose.github.io/rl-alignment-veltech-NB-lecture/days/day-05/lab.html#pd.

Interactive lab, Part E.

A win rate under both rules for ties and its half-width by hand, with other numbers than the worked examples. Part E opens at https://utkukose.github.io/rl-alignment-veltech-NB-lecture/days/day-05/lab.html#pe.

Python code in the Colab notebook, Section 5.

Open Section 5 of the Colab notebook and run it. The loop calls `win_rate` with 25, 100, 400 and 1600 sentences for the two comparisons and prints each interval. To test whether a decision survives fresh sentences, call `win_rate` again with another value of its option `seed`.

A blind taste test of two coffees shows the interval at work. Each taster drinks both coffees without knowing which is which and prefers A, prefers B or finds no difference. Coffee A is slightly better: 45 percent of the tasters prefer it, 35 percent prefer B and 20 percent find no difference, so the true win rate of A is 0.45 + 0.5 × 0.20 = 0.55. The tasters come one after another, and after every taster the win rate is computed with its interval by the formula of this section.

Animation on the lecture page: A blind taste test of two coffees: the win rate of A over 1000 tasters with its 95 percent interval, the true win rate as a dashed line and the first moment at which the interval leaves out one half. Choose whether the coffees differ, play the tasters and start another run.

In the first run, the interval after 100 tasters, from 0.488 to 0.682, still contains one half. It leaves out one half for the first time after 125 tasters, and after 400 tasters it lies between 0.514 and 0.611. Over 1000 runs, the test is decided for A at 400 tasters in only 50.3 percent of them, as expected for the minimum sample size of 384 that the formula gives for a win rate of 0.55. When the coffees are equal, the interval at 400 tasters leaves out one half in 2.9 percent of the runs. A test that stops as soon as the interval leaves out one half, however, declares a difference in 31.2 percent of them. Looking at the interval after every taster and stopping when it looks decided multiplies the false decisions, so the number of tasters is fixed before the test begins.

Check your understanding (5). A win rate is 0.52 with a 95 percent interval from 0.47 to 0.57. What can be concluded?

A. A is better

B. B is better

C. The comparison is undecided

D. The judge is biased

Check your understanding (6). Model A wins 58 of 100 comparisons against model B, and there are no ties. What is the half-width of the 95 percent interval, and is the comparison decided?

A. About 0.049, decided

B. About 0.097, undecided

C. About 0.097, decided

D. About 0.49, undecided

Evaluating safety

Day 4 named two failures of an assistant that is aligned for safety: a harmful completion of a harmful request, and an over-refusal of a harmless request that only resembles a harmful one. Both are measured with the tools of this day, as shares of a fixed test set with intervals. A safety test set holds harmful requests and harmless requests that resemble them, such as the 200 and the 250 prompts of the XSTest suite [14]. It is kept out of training, as the gold reward was, because a model that is tuned on its own test set looks safer than it is. Red teaming keeps adding requests that make models fail [15, 16].

A judge marks every answer, and the two failure rates are shares of k failures in n requests, with the interval of the previous section. When no failure occurs, the interval formula gives no width, and the rule of three of the previous section gives an upper end:

$$\hat{p} = \frac{k}{n} \pm 1.96 \sqrt{\frac{\hat{p}\,(1-\hat{p})}{n}}$$

$$\hat{p} < \frac{3}{n} \quad \text{with 95 percent confidence when } k = 0$$

For counts of a few failures the formula is rough, and an interval that would reach below zero is cut at zero. The bootstrap of the notebook needs no such approximation.

> **Worked example: two models on one test set**
>
> A test set has 200 harmful and 250 harmless requests, and the counts are invented for the illustration. Model A completes 6 harmful requests and refuses 4 harmless ones. Model B completes 1 harmful request and refuses 21 harmless ones.
>
> For the harmful completions of A, 6 / 200 = 0.030, 0.030 × 0.970 = 0.0291, √(0.0291 / 200) = 0.0121 and 1.96 × 0.0121 = 0.024, so the interval runs from 0.006 to 0.054. For B, 1 / 200 = 0.005 with the half-width 1.96 × √(0.005 × 0.995 / 200) = 1.96 × 0.005 = 0.010, an interval from 0, cut at zero, to 0.015. The two intervals overlap: B looks safer, but the test set cannot separate the two models on harmful requests.
>
> For the over-refusals, A has 4 / 250 = 0.016 with the half-width 1.96 × √(0.016 × 0.984 / 250) = 1.96 × 0.0079 = 0.015, from 0.001 to 0.031. B has 21 / 250 = 0.084 with the half-width 1.96 × √(0.084 × 0.916 / 250) = 1.96 × 0.0175 = 0.034, from 0.050 to 0.118. These intervals do not overlap: B refuses clearly more harmless requests than A.

The example shows the two lessons of a safety evaluation. A model can lower one failure rate by raising the other, so a report gives both rates, each with its interval, the size of the test set and the judge. And a test set of a few hundred requests can show a clear difference in over-refusal while it cannot separate rare harmful completions, which need many more requests. The judge is a choice of its own: People disagree about borderline answers, and a language model that judges brings its own biases [2].

> **Interactive lab, Part F.**
>
> The formulas of this section solved step by step with numbers for a safety evaluation of two models: the failure rates, their intervals, the rule of three for zero failures and the comparisons. Sliders set the size of the test set and the counts of both failures, and a figure shows the intervals. Part F opens at https://utkukose.github.io/rl-alignment-veltech-NB-lecture/days/day-05/lab.html#pf.

Python code in the Colab notebook, Section 6.

Open Section 6 of the Colab notebook and run it. The function `rate_interval` computes a failure rate and its bootstrap interval from the counts of a test set, and the cell prints both rates of the two models of the worked example and the upper ends of the rule of three for several sizes of a test set.

Check your understanding (7). A model produces no harmful completion on 100 harmful requests of a test set. What can be claimed with 95 percent confidence?

A. The model is safe

B. Its rate of harmful completions is below about 0.03

C. Its rate of harmful completions is zero

D. Nothing at all

Your judge

The application of the day lets the reader choose the judge: one that counts calm words, one that prefers short sentences, one that punishes repetition or one that only checks the opening word. The table gives the score of each judge and the win rates of the two aligned models against the fine-tuned model, again over 400 comparisons with ties counted as half.

Judge	Score of a sentence	RLHF	DPO
`calm words`	Number of different calm words	0.902	0.974
`short`	Minus the number of words	0.504, undecided	0.289
`no repetition`	Number of different words minus the number of words	0.454	0.294
`good opening`	1 if the first word is `the`, `a` or `my`, otherwise 0	0.498, undecided	0.499, undecided

Both models win clearly under the first judge. Under the second, DPO loses with 0.289, the mirror image of the length judge, and RLHF is undecided. Under the third, both models fall below one half. Under the fourth, more than 99 percent of the comparisons are ties, because almost every sentence opens correctly and the judge cannot tell the answers apart.

The third judge shows how a judge can miss its purpose. Its score counts every repeated word, also the word `the`, so `the dog waited in the garden quietly` loses against `a bird waited in the garden`. The gold reward punishes only repeated content words, which are rare in all three models. DPO writes `the` twice in 89 percent of its sentences and the fine-tuned model in 44 percent, and the judge turns this habit into a defeat. A win rate means nothing without its judge.

Interactive lab, Part G.

Sixteen verdicts by you on eight pairs of sentences, each pair in both orders: The lab counts your verdicts for the first position, the pairs that you judged the same way in both orders and the win rate of the better sentences, and the part asks for a contest with a position bias by hand. Part G opens at https://utkukose.github.io/rl-alignment-veltech-NB-lecture/days/day-05/lab.html#pg.

Python code in the Colab notebook, Section 7.

Open Section 7 of the Colab notebook. Its first line is a switch, `JUDGE = "calm words"`. Replace the text by `short`, `no repetition` or `good opening`, or add a scoring function to the dictionary `MY_SCORES`, and run the section. The cell prints the win rate of each aligned model against the fine-tuned model with its interval and draws them as bars.

TokenWorld, stage 5: the project at the end of the week

The project has reached its last stage. The table follows TokenWorld through the week, from the exact optimum of four tokens to a language model aligned in two ways, each stage scored by a reward that the methods did not see.

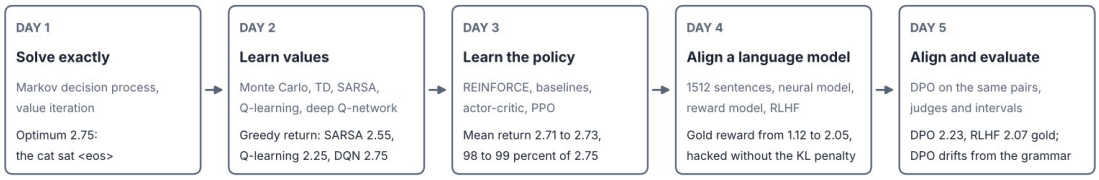


Figure. The five stages of TokenWorld through the week.

Day	World	Method	Result
1	4 tokens, 161 states, rules known	Value iteration	Exact optimum 2.75, the yardstick of the week
2	The same, rules unknown	SARSA (state, action, reward, state, action), Q-learning, deep Q-network	Greedy return 2.55 and 2.25; the network reaches 2.75
3	The same, policy learned	REINFORCE, actor-critic, proximal policy optimisation	98 to 99 percent of the optimum
4	1512 sentences, neural language model	SFT, reward model, RLHF	Gold reward from 1.12 to 2.05; hacked without the KL penalty
5	The same model and pairs	DPO, judges, intervals	Gold reward 2.23 with drift from the grammar; results stated with their judge and interval

Three habits carried the project from the first day to the last: Every result was measured against a reference that the method did not optimise, every claim came with its spread over seeds or its interval, and every shortcut that a learner found, from the cliff edge to the repeated word and the drifting grammar, was looked at in the actual outputs. The final project asks for the same habits on a world of each student's own.

The live dashboard. Section 8 of the Colab notebook gathers the project on one page. It repeats one run of Q-learning and one run of REINFORCE with a baseline on the small TokenWorld and passes their tables and the four networks of Days 4 and 5 to a web page in a frame of its own. The page evaluates them in the browser: value iteration and the exact returns of the small world, and the language models on all 1512 sentences of the grammar, with exact win rates of the judges over these sentences. Python computes the same numbers, and a fingerprint of them shows whether the page and Python agree. The last tab of the page scores any sentence with every model of the week.

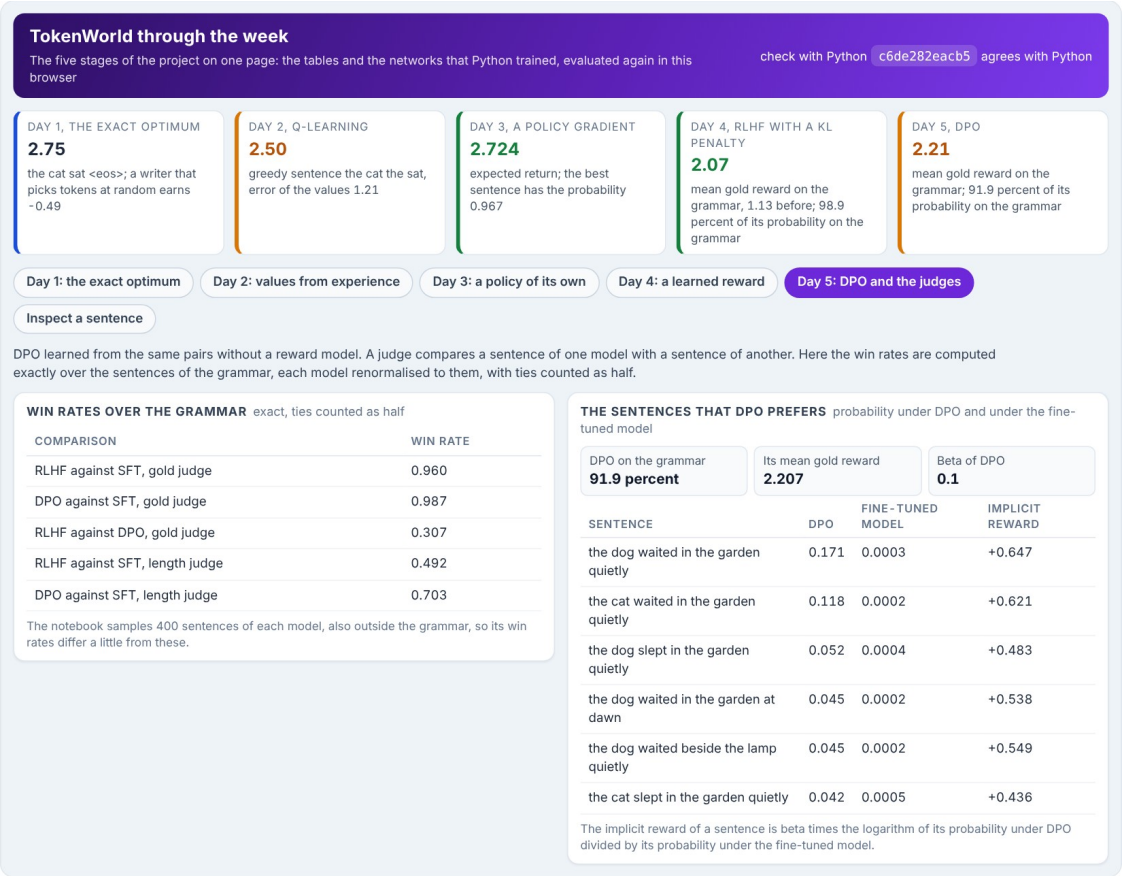


Figure. The live dashboard of Section 8 of the Colab notebook with its tab for Day 5: the five stages of TokenWorld in five tiles, the exact win rates of the judges over the 1512 sentences of the grammar and the sentences that DPO prefers.

> **Python code in the Colab notebook, Section 8.**
>
> Open Section 8 of the Colab notebook and run it after Sections 1 to 3. The function `tokenworld_dashboard` trains Q-learning and REINFORCE on the small TokenWorld, collects the fine-tuned model, the reward model, the RLHF model and the DPO model and shows the page below the cell. The cell prints the numbers that the page computes again and their fingerprint, and the header of the page shows its own fingerprint and whether the two agree.

Open problems and the final project

Alignment by preferences leaves open questions: whose preferences are collected, how judges can be checked, how a model behaves outside the data it was tuned on, and how to detect reward hacking that a learned reward cannot see [17]. Reward hacking, the subject of Day 4, is the case in which a policy raises a learned reward by exploiting its weaknesses while the quality of its answers falls [18, 19].

The week has shown several ways in which an alignment result can fail, and a safeguard against each. A learned reward can be hacked, which a gold reward that is kept out of training reveals. A policy can drift from its reference, which the KL divergence and a direct check of the outputs reveal. A judge can respond to a surface property or share the reward of the training, which a length control and an independent judge reveal. A win rate can depend on the rule for ties, on the sample or on the seed, which a stated rule, intervals and several seeds guard against.

The final project asks every student for a small alignment study that takes a world of its own along the path of TokenWorld, with every element of the week. Its brief asks for a gold reward that is written down and kept out of training, and for preference pairs from a stated annotator. Two alignment methods are compared on matched data and matched compute, which means the same pairs and a similar amount of computation. At least one method is pushed until the gold reward stops improving, the overoptimisation study of Day 4. The evaluation reports intervals, a length control and an independent judge, and the report discusses what the gold reward does not capture.

> **Python code in the Colab notebook, Section 9.**
>
> Open Section 9 of the Colab notebook and read the brief of the final project and its rubric. This section has no code to run.

Answers to the knowledge checks

Quick check. The reference would be the same object as the policy and change with it, so the KL penalty would always be zero and the leash of Day 4 would disappear.

1. C. DPO learns from the pairs directly; it needs neither a reward model nor sampling.

2. B. The margin is 1 minus 3, that is minus 2: The policy has raised the loser more than the winner. The weight is $\sigma(-\beta\, z) = \sigma(-0.5 \times (-2)) = \sigma(1) = 1 / (1 + e^{-1}) = 1 / (1 + 0.368) = 0.73$.

3. C. A single number hides the trade-off.

4. B. A length bias makes length look like quality.

5. C. The interval contains one half.

6. B. The half-width is $1.96 \sqrt{(0.58 \times 0.42 / 100)} = 1.96 \sqrt{(0.002436)} = 1.96 \times 0.0494 = 0.097$. The interval from $0.58 - 0.097 = 0.483$ to $0.58 + 0.097 = 0.677$ contains one half.

7. B. By the rule of three the rate lies below $3 / 100 = 0.03$. A rate of zero on 100 requests is compatible with a true rate of 1 or 2 percent.

References

[1] Rafailov, R., Sharma, A., Mitchell, E., Manning, C. D., Ermon, S., & Finn, C. (2023). Direct preference optimization: Your language model is secretly a reward model. In Advances in Neural Information Processing Systems 36 (NeurIPS 2023) (pp. 53728-53741).

[2] Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., & Stoica, I. (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track.

[3] Singhal, P., Goyal, T., Xu, J., & Durrett, G. (2024). A long way to go: Investigating length correlations in RLHF. In First Conference on Language Modeling (COLM 2024). arXiv:2310.03716.

[4] Henderson, P., Islam, R., Bachman, P., Pineau, J., Precup, D., & Meger, D. (2018). Deep reinforcement learning that matters. In Proceedings of the AAAI Conference on Artificial Intelligence, 32(1), 3207-3214.

[5] Van Rossum, G., & Drake, F. L. (2009). Python 3 Reference Manual. CreateSpace.

[6] Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., Fernández del Río, J., Wiebe, M., Peterson, P., Gérard-Marchant, P., Sheppard, K., Reddy, T., Weckesser, W., Abbasi, H., Gohlke, C., &

Oliphant, T. E. (2020). Array programming with NumPy. Nature, 585(7825), 357-362. https://doi.org/10.1038/s41586-020-2649-2

[7] Christiano, P. F., Leike, J., Brown, T. B., Martic, M., Legg, S., & Amodei, D. (2017). Deep reinforcement learning from human preferences. In Advances in Neural Information Processing Systems 30 (NeurIPS 2017) (pp. 4299-4307).

[8] Stiennon, N., Ouyang, L., Wu, J., Ziegler, D. M., Lowe, R., Voss, C., Radford, A., Amodei, D., & Christiano, P. F. (2020). Learning to summarize with human feedback. In Advances in Neural Information Processing Systems 33 (NeurIPS 2020) (pp. 3008-3021).

[9] Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P. F., Leike, J., & Lowe, R. (2022). Training language models to follow instructions with human feedback. In Advances in Neural Information Processing Systems 35 (NeurIPS 2022) (pp. 27730-27744).

[10] Korbak, T., Perez, E., & Buckley, C. (2022). RL with KL penalties is better viewed as Bayesian inference. In Findings of the Association for Computational Linguistics: EMNLP 2022 (pp. 1083-1091).

[11] Bradley, R. A., & Terry, M. E. (1952). Rank analysis of incomplete block designs: I. The method of paired comparisons. Biometrika, 39(3/4), 324-345.

[12] Azar, M. G., Guo, Z. D., Piot, B., Munos, R., Rowland, M., Valko, M., & Calandriello, D. (2024). A general theoretical paradigm to understand learning from human preferences. In Proceedings of the 27th International Conference on Artificial Intelligence and Statistics (AISTATS 2024), PMLR 238. arXiv:2310.12036. https://arxiv.org/abs/2310.12036

[13] Ethayarajh, K., Xu, W., Muennighoff, N., Jurafsky, D., & Kiela, D. (2024). KTO: Model alignment as prospect theoretic optimization. In Proceedings of the 41st International Conference on Machine Learning (ICML 2024). arXiv:2402.01306.

[14] Röttger, P., Kirk, H. R., Vidgen, B., Attanasio, G., Bianchi, F., & Hovy, D. (2023). XSTest: A test suite for identifying exaggerated safety behaviours in large language models. arXiv preprint arXiv:2308.01263. https://arxiv.org/abs/2308.01263

[15] Ganguli, D., Lovitt, L., Kernion, J., Askell, A., Bai, Y., Kadavath, S., Mann, B., Perez, E., Schiefer, N., Ndousse, K., Jones, A., Bowman, S., Chen, A., Conerly, T., DasSarma, N., Drain, D., Elhage, N., El-Showk, S., Fort, S., Hatfield-Dodds, Z., Henighan, T., Hernandez, D., Hume, T., Jacobson, J., Johnston, S., Kravec, S., Olsson, C., Ringer, S., Tran-Johnson, E., Amodei, D., Brown, T., Joseph, N., McCandlish, S., Olah, C., Kaplan, J., & Clark, J. (2022). Red teaming language models to reduce harms: Methods, scaling behaviors, and lessons learned. arXiv preprint arXiv:2209.07858. https://arxiv.org/abs/2209.07858

[16] Perez, E., Huang, S., Song, F., Cai, T., Ring, R., Aslanides, J., Glaese, A., McAleese, N., & Irving, G. (2022). Red teaming language models with language models. arXiv preprint arXiv:2202.03286. https://arxiv.org/abs/2202.03286

[17] Casper, S., Davies, X., Shi, C., Gilbert, T. K., Scheurer, J., Rando, J., Freedman, R., Korbak, T., Lindner, D., Freire, P., Wang, T. T., Marks, S., Segerie, C.-R., Carroll, M., Peng, A., Christoffersen, P. J. K., Damani, M., Slocum, S., Anwar, U., Siththaranjan, A., Nadeau, M., Michaud, E. J., Pfau, J., Krasheninnikov, D., Chen, X., Langosco, L., Hase, P., Biyik, E., Dragan, A., Krueger, D., Sadigh, D., & Hadfield-Menell, D. (2023). Open problems and fundamental limitations of reinforcement learning from human feedback. Transactions on Machine Learning Research.

[18] Skalse, J., Howe, N. H. R., Krasheninnikov, D., & Krueger, D. (2022). Defining and characterizing reward gaming. In Advances in Neural Information Processing Systems 35 (NeurIPS 2022).

[19] Gao, L., Schulman, J., & Hilton, J. (2023). Scaling laws for reward model overoptimization. In Proceedings of the 40th International Conference on Machine Learning (ICML 2023), PMLR 202, 10835-10866.

Prof. Dr. Utku Kose

Full Professor, Department of Computer Engineering, Süleyman Demirel University, Isparta, Türkiye

Founding Director, AI Application and Research Center (YAZEM), Süleyman Demirel University

Head of the Computer Science Division, Department of Computer Engineering, Süleyman Demirel University

Additional affiliations: University of North Dakota (USA), Universidad Panamericana (Mexico City, Mexico), Vel Tech University (Chennai, India)

IEEE Senior Member, ACM Professional Member

ORCID 0000-0002-9652-6415 · utkukose.com · github.com/utkukose

utkukose@sdu.edu.tr · utku.kose@und.edu · ukose@up.edu.mx · utkukose@gmail.com

This course is updated in line with current developments in the field. First version: October 2026. Last update: October 2026.